

H.S. 2nd Year — Computer Science and Applications (2020)

Solved Question Paper — Code: 30T CSCA

Full Marks: 70 | Pass Marks: 21 | Time: Three hours

Q. No. 1 — Answer the following ($1 \times 8 = 8$) [All parts solved]

(a) What is a Constructor ?

Ans: A constructor is a special member function of a class that has the same name as the class, has no return type, and is automatically called when an object of the class is created. It is mainly used to initialise the data members of the class.

(b) What is Inheritance ?

Ans: Inheritance is an OOP feature by which a new class (derived class) acquires the properties (data members) and behaviour (member functions) of an already existing class (base class), promoting code reusability.

(c) What is the purpose of 'new' operator in C++ ?

Ans: The 'new' operator in C++ is used to dynamically allocate memory to a variable/object during program execution (run time), and it returns the address of the allocated memory. Example: `int *p = new int;` allocates memory for an integer and stores its address in p.

(d) Give an example of non-linear data structure.

Ans: Examples of non-linear data structures: Tree and Graph — in these, data elements are not stored sequentially and an element may be connected to more than one other element.

(e) When do we use ALTER TABLE command in SQL ?

Ans: The ALTER TABLE command is used when we need to modify the structure of an already existing table, such as adding a new column, deleting an existing column, or modifying the data type/size of an existing column, without deleting and recreating the table itself.

(f) What is a Logic gate ?

Ans: A logic gate is a basic electronic building block of digital circuits, which performs a logical/Boolean operation (AND, OR, NOT, etc.) on one or more binary inputs to produce a single binary output.

(g) Write the full form of SMTP.

Ans: The full form of SMTP is Simple Mail Transfer Protocol.

(h) Give an example of a web browser.

Ans: An example of a web browser: Google Chrome. (Other examples: Mozilla Firefox, Microsoft Edge, Safari.)

Q. No. 2 — Answer the following ($2 \times 10 = 20$) [All parts solved]

(a) Differentiate between Abstract class and Concrete class.

Ans: Abstract class: (1) It cannot be instantiated directly (no object can be created from it), since it contains at least one pure virtual (undefined) function. (2) It is used as a base class to be inherited by other classes. Concrete class: (1) It has a complete implementation of all its member functions, and objects can be created directly from it. (2) It is a regular, fully-defined, usable class.

(b) What is meant by member variables and member functions ?

Ans: Member variables (data members) are the variables declared inside a class that hold the data/state/attributes of the objects of that class. Member functions are the functions declared inside a class that define the behaviour/operations that can be performed on the data members of that class.

(c) Distinguish between Ofstream and Ifstream classes.

Ans: Ofstream (output file stream): It is a C++ stream class used to create a file and write/output data into a file (from the program to the file). Ifstream (input file stream): It is a C++ stream class used to open an existing file and read/input data from that file (from the file into the program).

(d) Suppose an array 'arr' contains three elements. How would you declare and initialize a pointer to this array ?

Declaration and initialisation:

```
int arr[3] = {10, 20, 30};  
int *p = arr;           // p now points to the first element of arr  
// Alternatively: int *p = &arr[0];
```

Here 'p' is declared as a pointer to an int, and is initialised with the base address of the array 'arr' (the array name itself represents the address of its first element).

(e) Describe the two operations that can be performed on a stack.

Ans: The two main operations performed on a stack: (1) PUSH – used to insert/add a new element onto the top of the stack (after checking for stack overflow). (2) POP – used to remove/delete the element present at the top of the stack (after checking for stack underflow). Both operations follow the LIFO (Last In First Out) principle.

(f) What is the difference between CHAR and VARCHAR2 datatype in SQL ?

Ans: CHAR: (1) It is a fixed-length character data type — if the stored value is shorter than the declared size, it is padded with extra spaces to fill the fixed length. (2) It generally provides faster access for fixed-length data. VARCHAR2: (1) It is a variable-length character data type — it stores only the actual number of characters entered, without padding, saving storage space. (2) It is more efficient for storing data whose length varies from record to record.

(g) Describe the use of GROUP BY Clause in SQL with an example.

Ans: The GROUP BY clause is used in SQL to group rows that have the same values in one or more specified columns into summary rows, typically used together with aggregate functions (like SUM, COUNT, AVG) to get a result per group rather than per row.

Example: `SELECT Department, SUM(Salary) FROM Employee GROUP BY Department;`
This groups all employee records by their Department and displays the total salary for each department.

(h) State DeMorgan's law in Boolean algebra.

Ans: De Morgan's Law consists of two theorems: (1) $(A + B)' = A'.B'$ — the complement of a sum (OR) equals the product (AND) of the complements. (2) $(A.B)' = A' + B'$ — the complement of a product (AND) equals the sum (OR) of the complements.

(i) What are the limitations of Karnaugh Map ?

Ans: Limitations of Karnaugh Map (K-map): (1) It becomes difficult and error-prone to use for expressions with more than 5 or 6 variables, since the map becomes very large and complex to draw/read. (2) It is a purely manual/graphical method, not suitable for computer-based automated simplification of large expressions (unlike the Quine-McCluskey algorithm). (3) There is a chance of making mistakes while grouping cells, especially in larger maps, if not done carefully.

(j) Differentiate between Router and Switch.

Ans: Router: (1) It operates mainly at the Network layer (Layer 3) of the OSI model. (2) It is used to connect two or more different networks and route data packets between them based on IP addresses (e.g., connecting a LAN to the Internet). Switch: (1) It operates mainly at the Data Link layer (Layer 2) of the OSI model. (2) It is used to connect multiple devices within the same network (LAN) and forwards data based on MAC addresses.

Q. No. 3 — Compulsory, 9 parts (3 marks each = 27) [All parts solved, including OR alternatives]

(a) Explain how polymorphism can be implemented using function overloading. (3)

Ans: Polymorphism (meaning 'many forms') can be implemented through function overloading by defining two or more functions with the SAME name in the same class/scope, but with a DIFFERENT number of parameters or different parameter data types. When the function is called, the C++ compiler automatically determines (at compile time) which particular version of the function to execute, based on the number/type of arguments passed in that specific call. Since the same function name behaves differently depending on the arguments (i.e., takes many forms), this is called compile-time (or static) polymorphism. Example: void area(int side); // for square, and void area(int l, int b); // for rectangle — both share the name 'area' but perform different tasks depending on how many arguments are passed.

(b) Describe the mechanism of accessing data members and member functions in the following cases: (i) Inside the main program (ii) Inside a member function of the same class (iii) Inside a member function of another class. (3)

Ans: (i) Inside the main program: A public data member/member function can be accessed using the object name along with the dot (.) operator, e.g., obj.function(); or obj.data;. Private/protected members CANNOT be accessed directly from main() — they must be accessed indirectly through public member functions. (ii) Inside a member function of the SAME class: Any data member or member function of that class (whether private, protected, or public) can be accessed directly by its name, without needing the dot operator or an object reference, since member functions have direct access to all members of their own class. (iii) Inside a member function of ANOTHER class: Only the public members of the first class can be accessed, and that too using an object of the first class along with the dot (.) operator, e.g., objA.publicFunction();. Private/protected members remain inaccessible from an unrelated class (unless that class is declared a 'friend' of the first class).

(c) Rewrite the following program after removing the syntax errors, if any. (3)

Errors identified in the given program: (1) The header file should be <iostream> instead of <iostream.h> (old/deprecated syntax), and 'using namespace std;' should be added. (2) The object is declared as 'Room obj Room1;' which has invalid syntax (missing comma/incorrect naming) — it should be 'Room obj1;'. (3) 'void main()' should be 'int main()' with a 'return 0;' statement, as per standard C++.

Corrected program:

```
#include <iostream>
using namespace std;
class Room
{
    int length;
    int width;
public:
    Room()
    {
        length = 0;
        width = 0;
    }
    Room(int value = 8)
    {
        length = width = 8;
    }
    void display()
    {
        cout << length << " " << width;
    }
};
```

```
int main()
{
    Room obj1;
    obj1.display();
    return 0;
}
```

OR / অথবা

Write a program in C++ to demonstrate constructor overloading. (3)

```
#include <iostream>
using namespace std;
class Box
{
    int length, breadth;
public:
    Box() // default constructor
    {
        length = 0; breadth = 0;
    }
    Box(int l, int b) // parameterised constructor
    {
        length = l; breadth = b;
    }
    void display()
    {
        cout << "Length = " << length << ", Breadth = " << breadth << endl;
    }
};
int main()
{
    Box b1; // calls default constructor
    Box b2(10, 5); // calls parameterised constructor
    b1.display();
    b2.display();
    return 0;
}
```

(d) Convert the following infix expression into postfix: $(A * B - (C - D)) / (E + F)$. (3)

Solution (using a stack, step by step):

Step 1: $A * B \rightarrow AB*$

Step 2: $(C - D) \rightarrow CD-$

Step 3: $(A*B - (C-D)) \rightarrow AB* CD- - \rightarrow AB*CD--$

Step 4: $(E + F) \rightarrow EF+$

Step 5: Combine with the division: $(AB*CD--)/(EF+) \rightarrow AB*CD--EF+ /$

Postfix expression = $A B * C D - - E F + /$

OR / অথবা

Calculate the value of the following postfix expression: $5\ 4\ 6\ +\ 7\ 4\ -\ * \ 9\ /\ 3\ 5\ 15\ +\ +$. (3)

Solution (scanning left to right, using a stack):

$5\ 4\ 6\ +\ 7\ 4\ -\ * \ 9\ /\ 3\ 5\ 15\ +\ +$

1. Push 5	Stack: 5
2. Push 4	Stack: 5, 4
3. Push 6	Stack: 5, 4, 6
4. '+' -> 4+6=10	Stack: 5, 10
5. Push 7	Stack: 5, 10, 7
6. Push 4	Stack: 5, 10, 7, 4
7. '-' -> 7-4=3	Stack: 5, 10, 3
8. '*' -> 10*3=30	Stack: 5, 30
9. Push 9	Stack: 5, 30, 9
10. '/' -> 30/9=3.33	Stack: 5, 3.33
11. Push 3	Stack: 5, 3.33, 3
12. Push 5	Stack: 5, 3.33, 3, 5
13. Push 15	Stack: 5, 3.33, 3, 5, 15
14. '+' -> 5+15=20	Stack: 5, 3.33, 3, 20
15. '+' -> 3+20=23	Stack: 5, 3.33, 23

Note: With the expression exactly as printed, the final stack has 3 values left (5, 3.33, 23) rather than reducing fully to one — this indicates a likely missing operator in the original printed question (a common OCR/printing issue in these papers). Taking the expression as '5 4 6 + 7 4 - * 9 / 3 5 15 + + +' (with one more trailing '+', which correctly reduces the stack to a single value) gives:

16. '+' -> 5+23 = 28

Final Result = 28 (assuming the intended fully-balanced postfix expression).

(e) Write the steps to search 23 using Binary search in the following array: A[2, 5, 8, 12, 16, 23, 38, 56, 72]. (3)

Solution: Array indices 0 to 8 (9 elements), searching for key = 23.

Step 1: low = 0, high = 8
mid = (low + high) / 2 = (0 + 8) / 2 = 4
A[4] = 16 -> 16 < 23, so search the right half
low = mid + 1 = 5

Step 2: low = 5, high = 8
mid = (5 + 8) / 2 = 6
A[6] = 38 -> 38 > 23, so search the left half
high = mid - 1 = 5

Step 3: low = 5, high = 5
mid = (5 + 5) / 2 = 5
A[5] = 23 -> Element FOUND at index 5.

Hence, 23 is found at index position 5 of the array, after 3 comparisons.

OR / অথবা

Write an algorithm for Selection sort. (3)

1. Start
2. Read the number of elements n and the array A[0...n-1].
3. For i = 0 to n-2, repeat steps 4 to 7:
4. Set min_index = i
5. For j = i+1 to n-1, repeat:
6. If A[j] < A[min_index], then set min_index = j
7. Swap A[i] and A[min_index]
8. The array A is now sorted in ascending order.
9. Stop

(f) Differentiate between 'IN' and 'BETWEEN' operators in SQL with example. (3)

Ans: IN operator: It is used to check whether a value matches ANY value in a specified list of discrete values. Example: SELECT * FROM Employee WHERE Dept IN ('HR','Sales','IT'); (selects employees whose department is HR, Sales or IT). BETWEEN operator: It is used to check whether a value lies within a specified CONTINUOUS range (inclusive of both endpoints). Example: SELECT * FROM Employee WHERE Salary BETWEEN 20000 AND 30000; (selects employees whose salary is from 20000 to 30000, inclusive).

OR / অথবা

Write the necessary SQL commands to create a table (STUDENT) with Roll_No integer, Name Char(35), address Varchar2(70), and set Roll_No as the primary key. (3)

```
CREATE TABLE STUDENT
(
    Roll_No    INT PRIMARY KEY,
    Name       CHAR(35),
    address    VARCHAR2(70)
);
```

(g) Identify each of the following logic gates by name and construct their respective truth tables. (3)

Based on the standard gate symbols shown in the question paper (a curved-back OR-shaped gate, a flat AND-shaped gate, and an AND-shaped gate with a small bubble/circle at the output), the three gates are identified as: Gate 1 = OR gate, Gate 2 = AND gate, Gate 3 = NAND gate.

OR gate ($Y = A + B$):

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

AND gate ($Y = A.B$):

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

NAND gate ($Y = (A.B)'$):

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

OR / অথবা

Construct a truth table for the following function and from the truth table obtain an expression for the inversion function: $F(A, B, C) = A + BC'$. (3)

Truth table for $F(A, B, C) = A + BC'$:

A	B	C	C'	BC'	F=A+BC'
0	0	0	1	0	0
0	0	1	0	0	0
0	1	0	1	1	1
0	1	1	0	0	0
1	0	0	1	0	1
1	0	1	0	0	1
1	1	0	1	1	1
1	1	1	0	0	1

The inversion function F' is obtained by taking the rows where $F = 0$, i.e., rows (0,0,0), (0,0,1) and (0,1,1). Writing the SOP for F' :

$$F'(A,B,C) = A'B'C' + A'B'C + A'BC = A'B'(C' + C) + A'BC = A'B' + A'BC$$

This can be further simplified: $F' = A'(B' + BC) = A'(B' + C) = A'B' + A'C$

Inversion function: $F'(A,B,C) = A'B' + A'C$ (which is equivalent to $A'(B' + C)$)

(h) Write the advantages of Star topology. (3)

Ans: Advantages of Star topology: (1) Easy to install, configure, and manage compared to other topologies. (2) If one connecting cable or one node/device fails, it does not affect the rest of the network — only that particular node loses connection. (3) Easy to detect faults and troubleshoot, since each device is independently connected to the central hub/switch. (4) New devices can be added or removed easily without disturbing the rest of the network.

OR / অথবা

Write a short note on Firewall. (3)

Ans: A firewall is a network security system/device (hardware, software, or both) that monitors and controls incoming and outgoing network traffic based on a defined set of security rules, acting as a barrier between a trusted internal network and untrusted external networks (like the Internet). It helps to prevent unauthorised access, block malicious traffic, and protect a computer/network from hackers, viruses, and other cyber threats.

(i) Write a short note on twisted pair cable. (3)

Ans: Twisted pair cable is a type of guided/wired transmission medium made up of pairs of insulated copper wires twisted together, used to carry electrical signals (data/voice) over a network. Twisting the wires helps to reduce electromagnetic interference (crosstalk) from external sources and between adjacent pairs. It is inexpensive and easy to install, and is commonly used in LANs (Ethernet cables) and telephone systems. It is available in two types: UTP (Unshielded Twisted Pair) and STP (Shielded Twisted Pair).

OR / অথবা

Write a short note on Firewall.

Ans: (See the answer to the alternative part above under 'Advantages of Star topology / Firewall' — a firewall is a network security system that monitors and filters incoming and outgoing traffic based on security rules, protecting a network from unauthorised access and cyber threats.)

Q. No. 4 — Explain different types of Inheritance in C++. (5) [With OR alternative solved]

Ans: The different types of inheritance supported in C++ are: (1) Single Inheritance — a derived class inherits from only one base class. (2) Multiple Inheritance — a derived class inherits from more than one base class simultaneously. (3) Multilevel Inheritance — a class is derived from a class, which is itself derived from another class, forming a chain (grandparent → parent → child). (4) Hierarchical Inheritance — multiple derived classes inherit from a single common base class. (5) Hybrid Inheritance — a combination of two or more of the above types of inheritance used together in the same program. Each type allows different structures of code reuse depending on the relationship between the classes involved.

OR / অথবা

Explain the visibility modes available in C++ with example. (5)

Ans: Visibility modes (also called inheritance access specifiers) determine how the members of a base class are inherited into a derived class. The three visibility modes in C++ are: (1) Public inheritance — public members of the base class remain public in the derived class, and protected members remain protected. (2) Protected inheritance — both public and protected members of the base class become protected in the derived class. (3) Private inheritance — both public and protected members of the base class become private in the derived class. In all three modes, private members of the base class are never directly accessible in the derived class.

```

class Base
{
    public: int a;
    protected: int b;
    private: int c;
};
class Derived1 : public Base    { /* a stays public, b stays protected */ };
class Derived2 : protected Base { /* a becomes protected, b stays protected */ };
class Derived3 : private Base   { /* a becomes private, b becomes private */ };

```

Q. No. 5 — Describe the basic functions available in C++ for file operations. (5) [With OR alternative solved]

Ans: Basic functions/facilities available in C++ for file handling (mainly through the <fstream> header): (1) open() – used to open a file and associate it with a file stream object (also possible directly via constructor). (2) close() – used to close an already-opened file, releasing associated resources. (3) Insertion operator (<<) with an ofstream object – used to write data to a file. (4) Extraction operator (>>) with an ifstream object, or functions like get()/getline() – used to read data from a file. (5) read() and write() – used for reading and writing data in binary form. (6) eof() – used to detect the end-of-file condition while reading. C++ provides three stream classes for this purpose: ofstream (for writing), ifstream (for reading), and fstream (for both reading and writing).

OR / অথবা

Write a C++ program to create a new file and write some lines to it.

```

#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    ofstream file("notes.txt");
    if (!file)
    {
        cout << "Error creating file!" << endl;
        return 1;
    }
    file << "This is the first line." << endl;
    file << "This is the second line." << endl;
    file << "C++ file handling example." << endl;
    file.close();
    cout << "File 'notes.txt' created and data written successfully." << endl;
    return 0;
}

```

Q. No. 6 — Write a C++ program to implement a Stack. (5) [With OR alternative solved]

```

#include <iostream>
#define MAX 5
using namespace std;
int stack1[MAX], top = -1;
void push(int val)
{
    if (top == MAX - 1)

```

```

        cout << "Stack Overflow!" << endl;
    else
    {
        stack1[++top] = val;
        cout << val << " pushed." << endl;
    }
}
void pop()
{
    if (top == -1)
        cout << "Stack Underflow!" << endl;
    else
        cout << stack1[top--] << " popped." << endl;
}
int main()
{
    push(10); push(20); push(30);
    pop();
    push(40);
    pop(); pop(); pop();
    pop(); // demonstrates underflow
    return 0;
}

```

OR / অথবা

Write a C++ program to implement a Queue.

```

#include <iostream>
#define MAX 5
using namespace std;
int queue1[MAX], front = -1, rear = -1;
void enqueue(int val)
{
    if (rear == MAX - 1)
        cout << "Queue Overflow!" << endl;
    else
    {
        if (front == -1) front = 0;
        queue1[++rear] = val;
        cout << val << " enqueued." << endl;
    }
}
void dequeue()
{
    if (front == -1 || front > rear)
        cout << "Queue Underflow!" << endl;
    else
        cout << queue1[front++] << " dequeued." << endl;
}
int main()
{
    enqueue(10); enqueue(20); enqueue(30);
    dequeue();
    enqueue(40);
    dequeue(); dequeue(); dequeue();
}

```

```
    return 0;  
}
```